

CSX3007 (Computer Architecture) Assignment 05

- 1) The MIPS architecture has a register set that consists of **32-bit registers**. Is it possible to design computer architecture without a register set? If so, briefly describe the architecture, including the instruction set. What are the advantages and disadvantages of this architecture over the MIPS architecture?
- 2) Differentiate the **RISC** (Reduced Instruction Set Computer) and **CISC** (Complex Instruction Set Computer) architectures.
- 3) Briefly describe the **four state elements** of the **MIPS** system.
- 4) Show MIPS **single cycle data path** for the following *store instruction* (where **S0** = 0x00000004 and **S1** = 0xAF0563BC):

```
sw    S1, 0x0004(S0)
```

- 5) Consider the following MIPS assembly code (where S0, S1, and S2 are 32-bit general purpose registers, and S0 has initialized with a value of zero):

```
i)    addi  S0, S0, 7
ii)   sw    S1, 8(S0)
iii)  addi  S0, S0, 4
iv)   lw    S2, 4(S0)
```

5.1) Show the **immediate value** (in hexadecimal) in register **S0** after the execution of the instructions (i) and (iii).

5.2) Show the **32-bit physical memory addresses** (in hexadecimal) of the instructions (ii), and (iv).

- 6) Consider the following **MIPS instruction code** (where S0, S1, and S6 are 32-bit general purpose registers and S0 has initialized with zero):

```
i)      addi  S0, S0, 5
ii)     lw    S6, 5(S0)
iii)    sw    S1, 8(S0)
```

6.1) Show the **immediate value** in register **S0** after executing the instruction (i).

6.2) Show the **physical addresses** of the memory access instructions (ii and iii).

- 7) Assume that a user on an MIPS system creates the following (C/C++/Java) source code (where i, j, f, g, and h are integer variables):

```
if(i == j)
    f = g + h;
else
    f = f - 1;
```

Show its equivalent **assembly code**.

- 8) Consider the following MIPS instruction (where **S0** and **S1** are two 32-bit general-purpose integer registers):

```
sw    S1, 15(S0)
```

Assume that S0 is initialized with a value of **zero** and S1 has a 32-bit number, **0xFF223344**. Show the **byte-addressable memory** of the MIPS system after executing the store instruction.

- 9) Assume that a compiler (C/C++/Java) that is running on a MIPS system generates the following assembly code (where S0, S1, S2, S3, and S4 are 32-bit general-purpose registers):

```
        bne S3, S4, next
        add S0, S1, S2
        j  L1
next:
        sub S0, S0, S3
L1:
```

Show the **equivalent high-level code** for the assembly code.

- 10) Discuss the importance of **dynamic data segment** memory in MIPS systems (*your description should show how the memory sections of this segment support dynamic data allocation during run-time*).
- 11) Discuss the importance of **text segment** memory in MIPS systems.
- 12) Discuss the importance of **dynamic data segment** in MIPS systems.
- 13) What memory format is used by the MIPS system for its *load* and *store* operations?
- 14) In a MIPS system, the **program counter (PC)** increments by 4 after each instruction fetch during its *sequential execution*; why?
- 15) Consider a **32-bit word** which is stored at memory **word 4** in a **byte-addressable** memory;
- 15.1) What is the **byte address** of memory **word 4** (in hexadecimal)?
 - 15.2) Draw memory locations where a 32-bit number, **0xFF223344**, is stored at **word 4** in **big-endian** and **little-endian** formats.
 - 15.3) What memory format is used by the MIPS system for its load and store operations?
- 16) “A **32-bit** MIPS system needs a **5-bit** addressing mechanism to access its **register file**.” Point out the reason(s).
- 17) Consider memory storage of a **32-bit word** stored at memory **word 4** in a byte-addressable memory.
- 17.1) What is the byte address of memory word 4?
 - 17.2) What are the byte addresses that memory word **four** spans?
 - 17.3) Draw the number 0xFF223344 stored at word 4 in both **big-endian** and **little-endian** machines (your drawing should clearly label the byte address corresponding to each data byte value).